

# BANCO DE DADOS II: TRANSAÇÕES, CONCORRÊNCIA E RECUPERAÇÃO

*Guia Avançado de Estudos e Resolução de Atividades*

**Resumo**— Este documento consolida os fundamentos de transações em Banco de Dados, detalhando as propriedades ACID, estados transacionais, técnicas de escalonamento concorrente (com resolução de anomalias como atualização perdida e leitura suja) e mecanismos de recuperação de falhas via Logs e Checkpoints. Além da fundamentação teórica, apresenta-se a resolução rigorosa e passo a passo das atividades propostas.

## PARTE I: TRANSAÇÕES E PROPRIEDADES ACID

---

### 1. As Propriedades ACID

Para garantir a integridade dos dados, o Sistema Gerenciador de Banco de Dados (SGBD) assegura quatro propriedades fundamentais às transações:

- **Atomicidade (A):** O princípio do "tudo ou nada". Uma transação deve ser tratada como uma unidade indivisível. Todas as suas operações são refletidas no banco ou nenhuma delas é. *Exemplo Bancário:* Transferência de R\$ 50 da Conta A para a B. Se o sistema debita de A e falha antes de creditar B, a atomicidade garante o desfazimento (rollback) do débito. Sem ela, o dinheiro "sumiria".
- **Consistência (C):** A execução isolada de uma transação deve levar o BD de um estado consistente a outro estado consistente, respeitando as regras de integridade. *Exemplo Bancário:* A soma total dos saldos do banco antes e depois de uma transferência entre clientes deve permanecer exatamente a mesma. Sem consistência, a lógica de negócio seria violada.
- **Isolamento (I):** A execução de uma transação não deve sofrer interferência de outras transações executando concorrentemente. Uma transação em andamento não revela seus estados parciais. *Exemplo Bancário:* Se T1 está transferindo fundos e T2 calcula o saldo total do banco, T2 não pode ler os dados no meio da transferência de T1. Sem isolamento, T2 leria um valor incorreto (leitura suja).
- **Durabilidade (D):** Uma vez que a transação é efetivada (commit), seus efeitos tornam-se permanentes, mesmo em caso de falha de sistema (crash). *Exemplo Bancário:* Após o caixa confirmar um depósito, se houver queda de energia, o saldo do cliente deve permanecer atualizado quando o sistema voltar. Sem durabilidade, dados confirmados seriam perdidos.

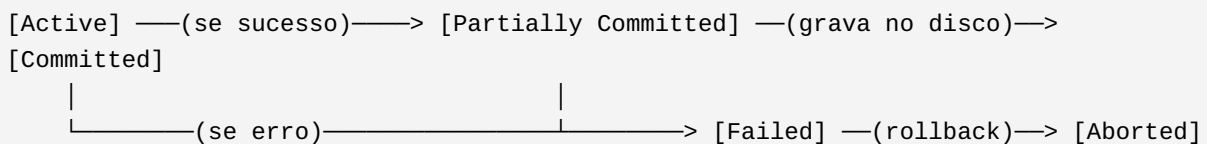
## 2. Cenário Acadêmico (Matrícula)

No cenário de matrícula (verificar vagas, reduzir vagas, registrar aluno):

- **a) Única transação:** O processo deve ser unificado para garantir que não se registre um aluno se a redução de vagas falhar, e não se reduza vagas se o registro falhar. Elas possuem dependência lógica estrita.
- **b) Falha entre os passos:** Se falhar após reduzir a vaga e antes do registro, a instituição "perde" a vaga (fica presa no limbo) e o aluno fica sem a disciplina.
- **c) Propriedades mais importantes:** *Atomicidade* (para garantir que ou a vaga e a matrícula ocorrem juntas, ou nenhuma ocorre) e *Isolamento* (para impedir que dois alunos tentem pegar a mesma e última vaga ao mesmo tempo, gerando overbooking).

## 3. Estados de uma Transação

O ciclo de vida de uma transação passa por estados rigorosos:



- **Active (Ativa):** Estado inicial, as operações de leitura/escrita estão ocorrendo.
- **Partially Committed:** Após a execução da última instrução, mas os dados ainda estão em buffer (memória volátil).
- **Failed (Falha):** Quando descobre-se que a execução normal não pode prosseguir (erro de lógica, deadlock, falha de hardware).
- **Aborted (Abortada):** Após o desfazimento (rollback) da transação para garantir a atomicidade. Pode ser reiniciada ou morta.
- **Committed (Efetivada):** Sucesso absoluto; as alterações estão salvas de forma segura em armazenamento estável.

## PARTE II: CONCORRÊNCIA E ESCALONAMENTO

### 4. Escalonamento e Serialização

**a) Serial:** Transações executam sequencialmente, uma após a outra. T1 inicia e só quando acaba T2 pode começar. Não há paralelismo.

**b) Concorrente:** As operações de múltiplas transações são intercaladas no tempo da CPU.

**c) Serialização:** É a propriedade de garantir que o resultado de um escalonamento concorrente seja exatamente igual ao de alguma ordem de escalonamento puramente serial.

*Por que preferir concorrência?* Para maximizar o "Throughput" (vazão) e minimizar o tempo de resposta, mantendo o processador ocupado enquanto uma transação espera operações lentas de I/O (leitura de disco).

## 5. Problema da Atualização Perdida (Lost Update)

Dado  $A = 5000$ . T1:  $R(A)$ ,  $A = A - 1000$ ,  $W(A)$ . T2:  $R(A)$ ,  $A = A - 500$ ,  $W(A)$ .

- **a) Escalonamento anômalo:**

T1:  $R(A)$  [lê 5000]

T2:  $R(A)$  [lê 5000]

T1:  $W(A)$  [escreve 4000]

T2:  $W(A)$  [escreve 4500]

- **b) Saldo incorreto produzido:** R\$ 4.500 (O saque de R\$ 1000 de T1 foi sobreposto e perdido).

- **c) Saldo correto esperado:** R\$ 3.500 (Execução serial T1->T2 ou T2->T1).

- **d) Como o controle evita:** Utilizando bloqueios (Locks). Se T1 adquirir um Bloqueio Exclusivo em A, T2 seria forçado a esperar T1 realizar o  $W(A)$  e comitar antes de poder  $R(A)$ . T2, então, leria  $A = 4000$ .

## 6. Atualização Temporária (Leitura Suja / Dirty Read)

Ocorre quando T2 lê um dado que foi modificado por T1, mas T1 falha e sofre rollback. O valor lido por T2 nunca existiu validamente.

- **a) Escalonamento com rollback:**

T1:  $R(A)$ ,  $A = A - 100$ ,  $W(A)$ .

T2:  $R(A)$  [Lê o A subtraído por T1].

T1: FALHA -> Rollback.

T2: Usa o valor de A, mas ele é inválido.

- **b) Por que o valor é inválido:** Porque a transação T1 foi abortada e, pelas regras de atomicidade, é como se ela nunca tivesse ocorrido.

- **c) Nível de isolamento:** READ COMMITTED (Garante que só se leia dados de transações já efetivadas).

## 7. Análise Inconsistente (Resumo Incorreto)

Ocorre quando uma transação de leitura (T1) está computando agregações sobre múltiplas tuplas enquanto outra transação de atualização (T2) altera alguns desses dados simultaneamente.

- **a/b) Exemplo:** T1 vai somar as contas A e B (total=100). T1 lê A(50). Antes de ler B, T2 transfere 10 de B para A (A vira 60, B vira 40) e comita. T1 lê B(40). A soma de T1 será  $50 + 40 = 90$ , o que está incorreto (o total real é 100).

- **c) Resolução via bloqueios:** Se T1 utilizar Bloqueios Compartilhados (Shared Locks) em A e B (ou Bloqueio em Duas Fases - 2PL), T2 não poderá obter um Bloqueio Exclusivo em A ou B até que T1 libere os bloqueios ao finalizar a soma, garantindo consistência.

## 8 e 9. Conflitos e Grafos de Precedência

### RESOLUÇÃO DA ATIVIDADE 8

Escalonamento: R1(A), R2(A), W1(A), W2(A).

- **a) Conflitos:** R2(A) com W1(A) (T2 precede T1). W1(A) com W2(A) (T1 precede T2).
- **b) Grafo:** Aresta  $T2 \rightarrow T1$  e Aresta  $T1 \rightarrow T2$ . O grafo forma um ciclo.
- **c/d) Serializável? NÃO.** A existência de um ciclo no grafo de serialização prova que o escalonamento não é equivalente por conflito a nenhuma ordem serial (causaria atualização perdida).

### RESOLUÇÃO DA ATIVIDADE 9

Escalonamento: R1(X), W1(X), R2(X), W2(X), R1(Y), W1(Y).

- **a) Conflitos:** W1(X) conflita com R2(X) (T1 precede T2). W1(X) conflita com W2(X) (T1 precede T2). Não há operações de T2 no item Y.
- **b) Grafo:** Aresta  $T1 \rightarrow T2$ .
- **c) Serializável? SIM.** Não existem ciclos no grafo. É equivalente por conflito ao escalonamento serial  $T1 \rightarrow T2$ .

## 10. Equivalência de Conflito vs Equivalência de Visão

- **Equivalência de Conflito:** Dois escalonamentos são equivalentes se a ordem de quaisquer duas operações conflitantes (R/W, W/R, W/W) é a mesma em ambos. Se um grafo de precedência não tem ciclos, ele é conflito-serializável.
- **Equivalência de Visão:** É um conceito mais amplo. Dois escalonamentos são visão-equivalentes se cada transação lê o mesmo valor inicial, lê valores produzidos pelas mesmas transações, e a gravação final do dado é feita pela mesma transação em ambos. Um escalonamento pode ser visão-serializável mesmo sem ser conflito-serializável (geralmente envolvendo gravações cegas / blind writes).

## PARTE III: RECUPERAÇÃO DE FALHAS (RECOVERY)

---

### 1. Análise de Logs (REDO / UNDO)

Para recuperar o BD após a falha, aplicam-se regras baseadas na estabilidade das transações. Transações que possuem um COMMIT no log antes da FALHA precisam ser refeitas (**REDO**). Transações que iniciaram mas não possuem COMMIT devem ser desfeitas (**UNDO**).

#### LOG 1

START T1 → T1: A,100,150 → START T2 → T2: B,200,250 → COMMIT T1 → T2: C,300,350 → FALHA

- **a) REDO:** T1 (possui COMMIT).
- **b) UNDO:** T2 (não possui COMMIT).
- **c) Valores finais:** A=150 (mantido/refeito), B=200 (desfeito de 250 para 200), C=300 (desfeito de 350 para 300).

#### LOG 2

START T1 → T1: A,100,150 → START T2 → T2: B,200,250 → T1: A,150,90 → COMMIT T1 → T2: C,300,350 → T2: B,250,300 → FALHA

- **a) REDO:** T1 (possui COMMIT).
- **b) UNDO:** T2 (não possui COMMIT).
- **c) Valores finais:** A=90 (REDO refaz para 150, depois para 90). T2 sofre UNDO operando do final para o começo do log: B volta de 300 para 250, C volta para 300, B volta de 250 para 200. Finais: A=90, B=200, C=300.

### 2. Log de Sistema de Venda de Ingressos

Vagas Iniciais = 1000. Fat(Faturamento) = 0. T1 compra 2 (R\$ 100 cada). T2 cancela 1.

```
START T1
T1: Vagas, 1000, 998
T1: Fat, 0, 200
COMMIT T1
START T2
T2: Vagas, 998, 999
T2: Fat, 200, 100
COMMIT T2
```

### 3. Checkpoints na Recuperação

O CHECKPOINT descarrega os buffers alterados para o disco e registra as transações ativas naquele instante. Isso evita que o sistema precise analisar todo o log desde o início dos tempos em caso de falha.

#### ANÁLISE DO LOG 1

A falha ocorre com T1 comitado, T2 não comitado, T3 não comitado.

- **a) Ativas na falha:** T2 e T3.
- **b) Necessitam REDO:** T1 (começou antes do checkpoint, mas comitou antes da falha).
- **c) Necessitam UNDO:** T2 e T3.

#### ANÁLISE DO LOG 2

Checkpoint lista [T1, T2] ativas. T0 comitou antes do checkpoint. T1 comitou após. T3 iniciou após. T2 e T3 estavam ativas na falha.

- **a) Ativas na falha:** T2 e T3.
- **b) Necessitam REDO:** T1 (comitado após checkpoint). *Nota: T0 já teve suas páginas garantidas no disco pelo Checkpoint.*
- **c) Necessitam UNDO:** T2 e T3.
- **d) Redução de trabalho:** O checkpoint garante que qualquer alteração de T0 (que comitou antes) já está fixa em armazenamento estável. O log scan da fase REDO pode começar no último checkpoint, economizando I/O imenso.

#### 4. Atuação do Sistema após a Falha

O Log mostra T1 efetivada, T2 e T3 interrompidas por falha. O algoritmo de recuperação (ex: ARIES) passa por três fases:

1. **Fase de Análise:** Escaneia o log e identifica as listas. `Lista_REDO = [T1]`. `Lista_UNDO = [T2, T3]`.
2. **Fase de Refazimento (REDO):** Aplica as alterações no banco simulando o histórico exato para restabelecer o estado até o travamento. (A=120, B=70, C=40, D=100).
3. **Fase de Desfazimento (UNDO):** Lê o log de trás para frente, revertendo as alterações das transações perdedoras. T3 reverte D(80). T2 reverte C(30) e B(50). Cria registros CLR no log para marcar a reversão.

**Valores Finais Pós-Recuperação:** A=120, B=50, C=30, D=80.